

Section 11.9 of 10: AI Skills, Tool Use & LLM Gateways

Executive Overview

Skills and gateways are the infrastructure layer between AI models and the tools/APIs they call. Tool-use mechanics govern *how* models select and invoke functions; gateways govern *which* models get called, at what cost, and under what policy. Understanding tool schema design, selection accuracy degradation, and multi-tenant cost control is essential for production AI systems.

11.9.1 Function Calling / Tool-Use Mechanics

Schema-Conditioned Tool Injection

Tools are injected into the model's context as JSON schemas. The model learns to emit structured JSON matching the schema when it wants to use a tool.

```
{
  "model": "claude-sonnet-4-6",
  "messages": [{"role": "user", "content": "Find customers over 30 in California"}],
  "tools": [
    {
      "name": "search_customers",
      "description":
        "Search the customer database. Use this when the user asks to find, filter, or list customers. NOT for analytics aggregations – use query_analytics for that.",
      "input_schema": {
        "type": "object",
        "properties": {
          "min_age": {"type": "integer", "minimum": 0, "description": "Minimum age filter"},
          "max_age": {"type": "integer", "maximum": 150},
          "state": {"type": "string", "description": "US state abbreviation, e.g. CA"},
          "limit": {"type": "integer", "default": 10, "maximum": 100}
        },
        "required": []
      }
    }
  ]
}
```

Model response with tool call:

```
{
  "role": "assistant",
  "content": null,
  "tool_calls": [{
    "id": "call_abc",
    "type": "function",
    "function": {
      "name": "search_customers",
      "arguments": "{\"min_age\": 30, \"state\": \"CA\", \"limit\": 10}"
    }
  }]
}
```

Structured Decoding (Grammar-Constrained Output)

Without constraints, models sometimes produce malformed JSON (missing brackets, wrong types).
JSON mode forces valid output:

Without JSON mode:

Model generates: {name: "search_customers", arguments: {query: "California customers over 30"...

→ Parser error: unquoted keys, wrong schema

With JSON mode / structured outputs:

Model output guaranteed to be valid JSON matching schema

→ Zero parsing errors in production

Providers:

OpenAI: response_format={"type": "json_schema", "json_schema": {...}}

Anthropic: tool_choice={"type": "any"} + schema validation

Ollama: format="json"

Tool Selection Accuracy vs. Tool Count

Empirical accuracy as tool count increases:

1-5 tools: 95% accurate selection

6-15 tools: 85% accuracy ← acceptable threshold

16-30 tools: 70% accuracy ← degraded

30+ tools: 55% accuracy ← unacceptable for production

Why it degrades:

- More schemas compete for attention
- Similar tool descriptions confuse model
- Schema context itself consumes tokens, reducing context for reasoning

Mitigation strategies:

Strategy	How It Works	Accuracy Recovery
Namespace partitioning	Group tools; route to relevant namespace first	85–90% with 30+ tools
Retrieval-based injection	Embed all tool descriptions; inject top-K at query time	90%+ even with 100+ tools
Tool descriptions quality	Clear differentiation, explicit “NOT for...” clauses	+5–10% accuracy
Few-shot examples	Show model 2–3 examples of correct tool selection	+5–8% accuracy

11.9.2 LLM Gateway Patterns

Multi-Model Routing

A gateway sits between the application and multiple LLM providers, routing requests based on capability requirements, cost, and availability.

```
class LLMGateway:
    ROUTING_RULES = {
        "vision": "gpt-4o",          # Requires multimodal
        "reasoning": "claude-sonnet-4-6", # Best reasoning
        "simple": "gpt-4o-mini",        # Cheap for basic tasks
        "code": "claude-sonnet-4-6",   # Best code quality
        "long_context": "claude-opus-4-6", # 200K context
    }

    def route_request(self, request: LLMRequest) -> str:
        # Check explicit capability requirements
        if request.has_images:
            return self.ROUTING_RULES["vision"]
        if request.context_tokens > 100_000:
            return self.ROUTING_RULES["long_context"]

        # Cost-based routing for simple queries
        complexity = estimate_complexity(request)
        if complexity < 0.3:
            return self.ROUTING_RULES["simple"]

        # Default to best capability model
        return self.ROUTING_RULES["reasoning"]
```

```
def route_with_fallback(self, request: LLMRequest) -> LLMResponse:
    primary = self.route_request(request)
    try:
        return call_model(primary, request)
    except RateLimitError:
        fallback = get_fallback(primary) # e.g., Claude → GPT-4o
        return call_model(fallback, request)
    except ModelUnavailableError:
        # Circuit breaker: mark model as down, reroute all traffic
        self.circuit_breaker.open(primary, duration_seconds=60)
        fallback = get_fallback(primary)
        return call_model(fallback, request)
```

Model Routing Decision Matrix

Request Type	Primary Model	Fallback	Cost Tier
Simple Q&A, classification	gpt-4o-mini / Haiku	Same tier	\$
Reasoning, planning	claude-sonnet-4-6	gpt-4o	\$\$\$ Vision + text gpt-4o claude-3-5-sonnet \$\$\$
Very long context (>100K)	claude-opus-4-6	gemini-1.5-pro	\$\$\$\$ Code generation claude- sonnet-4-6 gpt-4o \$\$\$
Batch/async (non-real-time)	Any, prefer cheapest	—	\$

Semantic Caching

```
class SemanticCache:
    SIMILARITY_THRESHOLD = 0.95 # Cosine similarity for cache hit

    def get(self, messages: List[dict]) -> CachedResponse | None:
        query_embedding = embed_messages(messages)
        similar = self.vector_store.search(
            query_embedding,
            k=1,
            min_similarity=self.SIMILARITY_THRESHOLD
        )
        if similar:
            self.metrics.increment("cache_hit")
            return similar[0].response
        return None

    def set(self, messages: List[dict], response: LLMResponse):
```

```

embedding = embed_messages(messages)
self.vector_store.upsert({
    "embedding": embedding,
    "response": response,
    "ttl": datetime.now() + timedelta(hours=24)
})

```

Cache hit rate economics:

Scenario: 1M requests/day, avg 5,000 input tokens
 50% semantic cache hit rate:
 Without cache: $1M \times 5K \text{ tokens} \times \$3.00/M = \$15,000/\text{day}$
 With 50% cache: $500K \times 5K \times \$3.00/M + 500K \times \sim\$0.001 \text{ (cache lookup)} = \$7,500 + \$500 = \$8,000/\text{day}$
 Savings: $\$7,000/\text{day}$ ($\$210,000/\text{month}$)

Rate Limiting & Cost Control

```

class TokenBudgetEnforcer:
    def check_and_charge(
        self,
        user_id: str,
        model: str,
        estimated_tokens: int
    ) -> Tuple[bool, str]:
        cost = self.pricing[model] * estimated_tokens / 1_000_000
        daily_spent = self.get_daily_spending(user_id)

        if daily_spent + cost > self.user_limits[user_id]["daily"]:
            return False, f"Daily budget exceeded. Spent: ${daily_spent:.2f},
Limit: ${self.user_limits[user_id]['daily']:.2f}"

        if cost > self.user_limits[user_id]["per_request"]:
            return False, f"Request too expensive: $
{cost:.4f} exceeds per-request limit"

        self.record_usage(user_id, cost)
        return True, "OK"

```

11.9.3 Gateway Architecture Comparison

Pattern	Throughput	Cost Control	Observability	Complexity	Best For
Direct API calls	High	None	None	Low	Prototypes

Pattern	Throughput	Cost Control	Observability	Complexity	Best For
Gateway with routing	Medium	Per-request	Request logs	Medium	Multi-model apps
Gateway + semantic cache	High	Excellent	Full	High	High-volume production
Multi-tenant gateway	Medium	Per-team budgets	Per-tenant dashboards	Very High	Enterprise, SaaS platforms

Interview Problems

Problem 1: Multi-Tenant LLM Gateway for 50 Teams

Scenario: 50 engineering teams share an LLM gateway. Each team has different cost sensitivity. Design budget enforcement with experimentation allowance.

Solution:

```
class MultiTenantGateway:
    def __init__(self):
        self.team_configs = load_team_configs() # From DB

    def handle_request(self, team_id: str, request: LLMRequest) -> LLMResponse:
        config = self.team_configs[team_id]

        # 1. Determine if this is an experimental request
        is_experimental = request.model not in config["approved_models"]

        # 2. Budget check
        if is_experimental:
            quota = config["experiment_quota"] # 10% of monthly budget
            if self.get_experiment_spend(team_id) > quota:
                raise BudgetExceeded("Experiment quota exhausted. Use approved
models.")
            else:
                if self.get_monthly_spend(team_id) > config["monthly_budget"]:
                    # Try cheaper fallback before failing
                    cheaper = self.get_cheaper_model(request.model)
                    if cheaper and config.get("allow_fallback", True):
                        request.model = cheaper
                else:
```

```

        raise BudgetExceeded("Monthly budget exhausted.")

    # 3. Route and call
    response = self.call_model(request)

    # 4. Record usage
    cost = calculate_cost(request.model, response.usage)
    self.record_spend(team_id, cost, is_experimental)

    return response

# Team tier examples:
TEAM_TIERS = {
    "startup": {"monthly_budget": 500, "approved_models": ["gpt-4o-mini"]},
    "business": {"monthly_budget": 5_000, "approved_models": ["gpt-4o", "claude-
haiku"]},
    "enterprise": {"monthly_budget": 50_000, "approved_models": ["*"]}, # All
models
    "research": {"monthly_budget": None, "approved_models": ["*"]}, #
Unlimited, separate billing
}

```

Observability dashboard metrics per team: - Daily/monthly spend vs. budget (% consumed) -
Model usage breakdown (which models, how much) - Cache hit rate (cost saved by caching) -
Budget burn rate (projected monthly total) - Alerts at 80% → warning email; 95% → page team lead

Problem 2: Tool Reliability in Production (Flaky External APIs)

Scenario: Your agent uses 5 tools, including a third-party weather API that has 2% failure rate and occasional 10-second timeouts. How do you make the agent resilient?

Solution:

```

class ResilientToolExecutor:
    def execute(
        self,
        tool_name: str,
        params: dict,
        config: ToolConfig
    ) -> ToolResult:
        for attempt in range(config.max_retries):
            try:
                result = self.tools[tool_name].call(
                    params,
                    timeout=config.timeout_seconds
                )
                return ToolResult(success=True, data=result)
            except TimeoutError:

```

```

        if attempt < config.max_retries - 1:
            time.sleep(2 ** attempt) # Exponential backoff: 1s, 2s, 4s
            continue
        # Final attempt failed: return degraded result
        return ToolResult(
            success=False,
            error="timeout",
            fallback_data=self.get_cached_result(tool_name, params)
        )

    except RateLimitError:
        # Don't retry – switch to backup provider
        return self.execute_on_backup(tool_name, params, config)

def execute_on_backup(self, tool_name, params, config):
    backup = self.backups.get(tool_name)
    if backup:
        return backup.call(params, timeout=config.timeout_seconds)
    return ToolResult(success=False, error="no_backup_available")

```

Tool reliability configuration:

```

TOOL_CONFIGS = {
    "weather_api": ToolConfig(
        timeout_seconds=3,
        max_retries=2,
        backoff="exponential",
        fallback="cached_weather" # Return last known weather
    ),
    "database_query": ToolConfig(
        timeout_seconds=5,
        max_retries=3,
        backoff="linear",
        fallback=None # No fallback – fail fast on DB errors
    ),
    "web_search": ToolConfig(
        timeout_seconds=8,
        max_retries=1,
        backoff="none",
        fallback="serp_backup" # Switch to backup search provider
    )
}

```

Problem 3: Debugging Tool Call Loops and Misrouting

Scenario: Your agent keeps calling `search_customers` when it should call `query_analytics`. Misrouting rate is 20%. How do you diagnose and fix it?

Solution:

Diagnosis:

1. Log all tool calls with context

Schema shows: `search_customers` = "Search customer records"

`query_analytics` = "Analyze customer metrics"

Confusion: Both seem relevant for "How many customers are in California?"

2. Inspect failing cases:

Query: "How many customers are over 30?"

Model selects: `search_customers` (wrong – returns rows, not count)

Correct: `query_analytics` (returns aggregated count)

Root causes:

a) Tool descriptions overlap: both mention "customers"

b) Model doesn't know `search_customers` returns rows vs. `query_analytics` returns aggregates

c) No examples showing which to use for counting queries

Fixes:

Fix 1 – Description differentiation:

`search_customers`: "Returns individual customer records (rows). Use for: find a specific customer, list customers matching criteria. NOT for counts or aggregations."

`query_analytics`: "Returns aggregated statistics. Use for: how many, what percentage, average age, total revenue. NOT for individual records."

Fix 2 – Add few-shot examples in system prompt:

Example 1: "How many customers are in CA?" → `query_analytics({metric: "count", filter: {state: "CA"}})`

Example 2: "Find John Smith's account" → `search_customers({name: "John Smith"})`

Fix 3 – Input type classifier (if descriptions alone aren't enough):

Detect aggregation intent: "how many", "total", "average", "percentage"

→ Pre-select `query_analytics`, inject only that tool schema

Detect record lookup intent: "find", "show", "get", "list specific"

→ Pre-select `search_customers`

Result: Misrouting drops from 20% to <3%